

Relatório de Implementação Ágil (RAD)

Mini Sistema de Controle de Estoque

Disciplina:	Desenvolvimento Rápido de Aplicações
Equipe:	Gabriel, Victoria, Paulo, Juan
Tecnologias:	Python 3.10+ · tkinter · SQLite3 · Faker
Repositório:	github.com/Gabriel07-gif/controle-estoque-rad

1. Decisões Rápidas

O principal objetivo da metodologia RAD é entregar um protótipo funcional no menor tempo possível, priorizando iterações rápidas sobre planejamento extenso. As decisões a seguir foram fundamentais para atingir esse objetivo:

- **Arquitetura modular com quatro arquivos independentes:** O projeto foi dividido em **banco.py** (persistência), **interface.py** (GUI), **utils.py** (validações e logs) e **principal.py** (integração). Essa separação permitiu que cada membro da equipe trabalhasse simultaneamente em seu módulo, sem conflitos de código, usando branches individuais no Git.
- **Bibliotecas nativas do Python:** Escolhemos **tkinter** e **sqlite3** porque já vêm instalados com o Python, eliminando configurações extras. Qualquer membro da equipe conseguiu rodar o projeto imediatamente após clonar o repositório, sem instalar frameworks pesados.
- **Contrato de funções definido antes da codificação:** Antes de iniciar o desenvolvimento, a equipe definiu os nomes, parâmetros e retornos de todas as funções (ex: **inserir_produto(nome, quantidade, preco) → bool**). Isso funcionou como uma "API interna" que permitiu ao responsável pela interface escrever chamadas de função antes mesmo do banco estar pronto.
- **Git com branches e Pull Requests:** O uso de branches nomeadas (**feature/banco-de-dados**, **feature/utils-validacoes**, **interface**) com Pull Requests no GitHub garantiu que cada contribuição fosse revisada antes de entrar na branch principal, reduzindo retrabalho e bugs de integração.

2. Estratégia de Teste

Para validar o sistema rapidamente, adotamos uma abordagem de testes manuais combinada com geração automática de dados:

- **População automática com Faker:** A função **popular_dados_teste()** utiliza a biblioteca **Faker** (com localização **pt_BR**) para gerar 10 produtos fictícios com nomes, quantidades (1–100) e preços aleatórios. Essa função só insere dados se a tabela estiver vazia, evitando duplicação. Com isso, qualquer membro da equipe podia testar

o sistema com dados realistas em segundos, sem digitar manualmente.

- **Testes manuais de CRUD:** Cada operação (cadastrar, editar, excluir) foi testada manualmente pela interface, verificando se o Treeview atualizava corretamente, se o log era gerado no formato esperado e se o banco de dados refletia as alterações. Erros de validação foram testados inserindo texto em campos numéricos, campos vazios e valores negativos.
- **Verificação do log de auditoria:** Após cada operação, o arquivo **log_estoque.txt** foi inspecionado para confirmar que o formato seguia o padrão exigido: **[DD/MM/AAAA HH:MM:SS] TIPO - Mensagem**. Ajustes foram feitos até que o log de ATUALIZAÇÃO incluísse o novo preço e o de EXCLUSÃO mostrasse o nome do produto ao invés de apenas o ID.

3. Visão de Futuro (Versão 2.0)

Seguindo o princípio RAD de iterações contínuas, as duas primeiras funcionalidades planejadas para o próximo ciclo de desenvolvimento são:

- **Sistema de alertas para estoque baixo:** Implementar um limiar configurável (ex: 5 unidades) que destaque em vermelho no Treeview os produtos com quantidade abaixo do mínimo e exiba uma notificação ao usuário ao abrir o sistema. Isso transformaria o software de um simples cadastro em uma ferramenta proativa de gestão, ajudando o lojista a evitar rupturas de estoque.
- **Relatórios gráficos de movimentação:** Adicionar uma seção na interface com gráficos (usando a biblioteca **matplotlib**) que mostrem o histórico de entradas e saídas por produto ao longo do tempo, além de um ranking dos produtos mais movimentados. Para isso, seria necessário criar uma tabela auxiliar de **movimentações** no banco de dados, registrando cada operação com data e tipo.

Conclusão

A metodologia RAD mostrou-se eficaz para o escopo deste projeto. A combinação de bibliotecas nativas, arquitetura modular e versionamento com Git permitiu que uma equipe de quatro pessoas entregasse um protótipo funcional com CRUD completo, persistência em banco de dados, log de auditoria e interface gráfica intuitiva em um ciclo curto de desenvolvimento. As melhorias propostas para a Versão 2.0 demonstram que o sistema possui uma base sólida para evoluções futuras.